UniSENAI

# Estruturas de Dados

**Aula 01:** Apresentação da disciplina, critérios de avaliação e recursos. Revisão prática de conceitos fundamentais da lógica de programação e da linguagem Java: tipos de dados, variáveis e constantes, estruturas de seleção e repetição, operadores, vetores e matrizes. Resolução de problemas clássicos com foco na construção do raciocínio algorítmico.

*Matheus Luan Krueger, M.e*

UniSENAI

# Roteiro

- Sobre a Unidade Curricular (UCr)
- Cronograma de aulas e sistema de avaliação
- Sobre o professor
- Sobre os alunos
- Revisão de conceitos de computação
- Revisão de conceitos de programação
- Problemas de lógica de programação em Java

UniSENAI

# Sobre a UCr

- O que já passou?

| Unidade curricular ↑↓ | Tipo | CH - UC | C.H - P | C.H - NP | C.H - MT | Cód. GR |
|---|---|---|---|---|---|---|
| 35479 - Algoritmos e Programação | Regular | 108 | 81 | 27 | | ALPR |
| 35481 - Arquitetura de Computadores e Sistemas Operacionais | Regular | 72 | 54 | 18 | | ACSO |
| 35484 - Comunicação Oral e Escrita | Regular | 36 | 1 | 35 | | COOE |
| 35482 - Fundamentos de Matemática para Informática | Regular | 36 | 27 | 9 | | FUMI |
| 35480 - Fundamentos de Segurança da Informação | Regular | 72 | 54 | 18 | | FUSI |
| 35477 - Metodologia de Projetos | Regular | 40 | 40 | 0 | | MEPR |
| 35483 - Sistemas de Inovação e Empreendedorismo | Regular | 36 | 1 | 35 | | SIIE |

UniSENAI

# Sobre a UCr

- O que já passou?

| Unidade curricular ↑↓ | Tipo | CH - UC | C.H - P | C.H - NP | C.H - MT | Cód. GR |
|---|---|---|---|---|---|---|
| 35896 - Eletiva I | Regular | 36 | 1 | 35 | | ELE1 |
| 35893 - Engenharia de Requisitos | Regular | 72 | 54 | 18 | | ENRE |
| 35895 - Probabilidade e Estatística | Regular | 36 | 1 | 35 | | PRES |
| 35892 - Programação Orientada à Objetos | Regular | 108 | 81 | 27 | | PROO |
| 35878 - Projeto Aplicado I (AEx) | Regular | 40 | 40 | 0 | | PRO1 |
| 35879 - Projeto, Desenvolvimento e Gerenciamento de Banco de Dados | Regular | 108 | 81 | 27 | | PDGB |

UniSENAI

# Sobre a UCr

- O que está por vir?

| Unidade curricular ↑↓ | Tipo | CH - UC | C.H - P | C.H - NP | C.H - MT | Cód. GR |
|---|---|---|---|---|---|---|
| 35898 - Arquitetura e Modelagem de Sistemas | Regular | 108 | 81 | 27 | | ARMS |
| 35905 - Desenvolvimento Web | Regular | 72 | 54 | 18 | | DEWE |
| 35902 - Estruturas de Dados | Regular | 72 | 54 | 18 | | ESDA |
| 35907 - Interface Humano-Computador | Regular | 36 | 27 | 9 | | INHC |
| 35897 - Projeto Aplicado II (AEx) | Regular | 40 | 40 | 0 | | PRO2 |
| 35908 - Tecnologia de Redes Locais | Regular | 72 | 2 | 70 | | TERL |

UniSENAI

# Sobre a UCr

- Ementa cf. PPC

| Módulo: ESPECÍFICO I | | | |
|---|---|---|---|
| **Unidade Curricular** | **Estruturas de Dados** | | |
| **Pré-requisito:** | | | |
| **Carga Horária:** | 72h | Presencial<br>54h | À distância<br>18h |
| **Função:** | F1 - Projetar sistemas computacionais, atendendo normas e padrões de qualidade, usabilidade, integridade e segurança da informação. | | |
| **Objetivo Geral:** | Propiciar o desenvolvimento de capacidades básicas e socioemocionais referentes às estruturas de dados, de forma a fundamentar a implementação e a execução de atividades de operação de sistemas computacionais. | | |

UniSENAI

# Sobre a UCr

- Ementa cf. PPC

| Conteúdos Formativos | | | |
|---|---|---|---|
| **Subfunção** | **Padrões de Desempenho** | **Capacidades Técnicas** | **Conhecimentos** |
| Levantar os requisitos | Considerando as regras e necessidades de negócio do cliente | Distinguir estruturas de dados; | • Estruturas de Dados<br>  ○ Linguagens de programação<br>  ○ Estruturas de dados homogêneas:<br>  ▪ Vetores e Matrizes<br>  • Estruturas de dados heterogêneas:<br>  • Registros<br>  • Lista<br>  • Fila<br>  • Pilha<br>  • Árvores<br>  • Tabelas Hash<br>  • Arquivos de dados:<br>  ▪ Binário<br>  ▪ Texto<br>  • Métodos de Pesquisa<br>  • Métodos de Ordenação |
| | | Implementar estruturas de dados | |
| | | Selecionar estruturas de dados de forma adequada para a resolução de problemas computacionais | |
| | | Utilizar arquivos de dados (binários e texto); | |
| | Considerando informações da necessidade do cliente/usuário e infraestrutura | Selecionar métodos de pesquisa; | |
| | | Selecionar métodos de ordenação; | |
| Codificar Sistemas | Considerando o fluxo definido do processo, de acordo com as especificações técnicas validadas; | Interpretar especificações técnicas; | |
| | Utilizando linguagem de programação, de acordo com projeto de sistema do software; | Utilizar linguagens de programação | |
| | | Distinguir erros no código fonte; | |
| | | Descrever o fluxo de processo. | |

UniSENAI

# Sobre a UCr

- Ementa cf. PPC

| Observações | |
|---|---|
| **Ambientes Pedagógicos** | AVA com recursos de interatividade; Sala de aula; Laboratório de informática; Biblioteca. |
| **Máquinas, Equipamentos, Instrumentos e Ferramentas** | Computadores ou notebooks; Softwares de aplicativos de escritório; Rede local, cabeada ou sem fio; Projetor multimídia; Sistemas Operacionais; Sistemas Distribuídos;; Bancos de Dados; Ambientes de Desenvolvimento Integrados; Ferramentas de Modelagem de Sistemas, Processo de Software, Segurança da Informação, Gestão de Projetos; Ambientes para Computação em Nuvem; Dispositivos Móveis; Ferramentas de prototipagem, monitoramento, integração e versionamento; Plataformas de gamificação; Simuladores da área do curso. |
| **Recursos Didáticos** | Livros impressos e/ou digitais; Recursos de aprendizagem em mídias digitais; Fóruns e repositórios do setor; Publicações do setor (revistas técnicas, artigos técnicos, manuais e catálogos de produtos e serviços, anais de congressos e sites especializados); Internet; Legislações trabalhistas, ambientais, de saúde e segurança, etc. Normas técnicas e padrões da área. |
| **Observações / Recomendações** | Serão asseguradas as condições de acessibilidade, reconhecendo a especificidade e a peculiaridade do aluno com deficiência, levando-se em conta a(s) Norma(s) Regulamentadora(s) da ocupação, Lei nº 13.146/2015, o Decreto nº 3298/2009, a LDM nº 9304/96 e a legislação específica em vigência da deficiência em questão, quando for o caso. Portanto, no planejamento e na prática docente, deverão ser indicadas as condições e os pré-requisitos para o desenvolvimento das capacidades que envolvam risco, asseguradas as adequações de grande e pequeno porte. |

UniSENAI

# Cronograma de aulas e sistema de avaliação

- **Parte 1: revisando programação e analisando comportamento**

| | |
|---|---|
| **29/07** | Apresentação da disciplina, critérios de avaliação e recursos. Revisão prática de conceitos fundamentais da lógica de programação e da linguagem Java: tipos de dados, variáveis e constantes, estruturas de seleção e repetição, operadores, vetores e matrizes. Resolução de problemas clássicos com foco na construção do raciocínio algorítmico. |
| **05/08** | Revisão e aprofundamento sobre modularização: definição de **métodos**, passagem de parâmetros, escopo e retorno de valores. Manipulação de vetores e matrizes em Java, com ênfase na leitura, escrita e iteração sobre estruturas homogêneas de dados. Introdução à **recursividade**: estrutura, funcionamento e exemplos clássicos (fatorial, sequência de Fibonacci, torres de Hanói). |
| **12/08** | Recursividade: fechamento. Comparação entre abordagens iterativas e recursivas. Introdução à **análise de complexidade de algoritmos**, com foco em notações assintóticas (O, Ω, Θ). |
| **19/08** | **Prova teórico-prática individual com questões contextualizadas, abordando conceitos teóricos de lógica, modularização, recursividade e análise de complexidade. AV01** |

UniSENAI

# Cronograma de aulas e sistema de avaliação

- **Parte 2: desenvolvendo estruturas de dados**

| | |
|---|---|
| **26/08** | Introdução à programação orientada a objetos aplicada a estruturas de dados. Adoção das classes ArrayList e LinkedList (e seus respectivos métodos). Apresentação e definição da interface Lista, estabelecendo um contrato para diferentes tipos de implementação. Introdução ao conceito de generalização de classes e uso de polimorfismo para manipular listas contíguas de forma abstrata. |
| **09/09** | Implementação da estrutura de **lista** utilizando vetores como base de armazenamento. Criação de métodos para inserção, remoção e busca. Encerramento da implementação da estrutura de **lista** com alocação de memória **contígua**. Discussão sobre as vantagens e limitações da **implementação contígua**. |
| **16/09** | Implementação da estrutura de **lista** com **alocação dinâmica**, utilizando nós encadeados. Criação de uma classe No e métodos básicos de manipulação. |
| **23/09** | Encerramento da implementação da estrutura de **lista** com alocação de memória **dinâmica**. Comparativo com a implementação contígua, destacando aspectos de desempenho e flexibilidade. |
| **30/09** | **Atividade prática de desenvolvimento de estrutura de dados baseada em listas (contígua ou dinâmica), avaliando o domínio da orientação a objetos e a capacidade de aplicar/implementar métodos definidos pela interface Lista. AV02** |

UniSENAI

# Cronograma de aulas e sistema de avaliação

- **Parte 2: desenvolvendo estruturas de dados**

| | |
|---|---|
| **07/10** | Implementação da estrutura de **fila** considerando diferentes formas de manipulação da memória, como alocação contígua e dinâmica. Estudo das operações de enfileiramento e desenfileiramento, além da introdução à fila circular. Discussão sobre aplicações clássicas de filas em algoritmos e sistemas computacionais. |
| **14/10** | Implementação da estrutura de **pilha** considerando diferentes formas de manipulação da memória, como alocação contígua e dinâmica. Estudo das operações de empilhamento, desempilhamento e inspeção do topo. Aplicações práticas envolvendo avaliação de expressões e conversão entre notações infixa e pós-fixa. |
| **21/10** | Conceito de espalhamento (hashing). Implementação de **tabela hash** com resolução de colisões (endereçamento aberto e encadeamento). Aplicações práticas, vantagens e desvantagens da estrutura. Estudo introdutório das estruturas de **árvores**, com foco em árvores binárias e extensão para árvores n-árias. |
| **28/10** | **Avaliação teórica individual com foco nos conceitos, estruturas, vantagens/desvantagens e aplicações das TADs: lista (contígua e dinâmica), fila, pilha, árvore e hash. AV03** |

UniSENAI

# Cronograma de aulas e sistema de avaliação

- **Parte 3: desenvolvendo algoritmos**

| | |
|---|---|
| **04/11** | Implementação e análise dos **algoritmos de ordenação** Bubble Sort e Insertion Sort. Apresentação da complexidade de tempo e espaço. Casos melhores, médios e piores. Comparações gráficas de desempenho. |
| **11/11** | Implementação e análise dos algoritmos Selection Sort e QuickSort. Exploração do pivô e divisão de listas. Comparações com os métodos anteriores. |
| **18/11** | Algoritmos de ordenação baseados em divisão e conquista. Merge Sort (recursividade e estabilidade). Início alg busca, |
| **25/11** | Exploração dos **algoritmos de busca**, incluindo busca linear, binária, ternária e por salto. A aula abordará suas implementações, diferenças de desempenho, complexidade computacional e condições ideais de uso — como a necessidade de vetores ordenados. Serão apresentados exemplos práticos e discutidas as limitações e aplicabilidades de cada abordagem. |
| **02/12** | **Revisão seguida de realização de atividade prática com foco em algoritmos de ordenação e busca, envolvendo análise comparativa de estruturas e algoritmos quanto à eficiência e aplicabilidade. AV04** |

UniSENAI

# Cronograma de aulas e sistema de avaliação

- **Parte 4: encerrando os trabalhos**

| | |
|---|---|
| **03/12** | **Prazo para o lançamento das notas do Caderno de Atividades (AV05) e da AIC 2026/2 (AV06).** |
| **09/12** | **Exame final: avaliação destinada para alunos com média pertencente ao intervalo [3,00 , 6,00) e com frequência mínima de 75%.** |

- **Frequência mínima para aprovação: 75%**
- **Sistema de avaliação:**
  - AV01: peso de 15,00% 19/08
  - AV02: peso de 20,00% 30/09
  - AV03: peso de 20,00% 28/10
  - AV04: peso de 15,00% 02/12
  - AV05: peso de 20,00% (***Caderno de ativ.***) 03/12
  - AV06: peso de 10,00% (***AIC 2026/2***) 03/12

UniSENAI

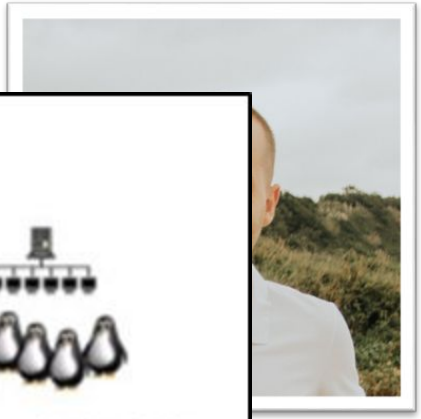
# Sobre o professor



- **Matheus Luan Krueger**
  - Técnico em Manutenção de Microcomputadores e Redes pelo Serviço Nacional de Aprendizagem Industrial de Santa Catarina (2007)
    - *Implantação de servidor LTSP para terminais remotos*
  - Bacharel em Ciência da Computação pela Fundação Universidade Regional de Blumenau (2012)
    - *Protótipo de software para autenticação biométrica baseada na estrutura da íris em dispositivos móveis*
  - Mestre em Computação pela Universidade Federal do Rio Grande do Sul (2015)
    - *Visual Simulation of Refractive Errors on Monochromatic Images and An Evaluation of the Absolute Threshold for Vision*

UniSENAI

# Sobre o professor



- **Ma**
  - 
  - 
  - 

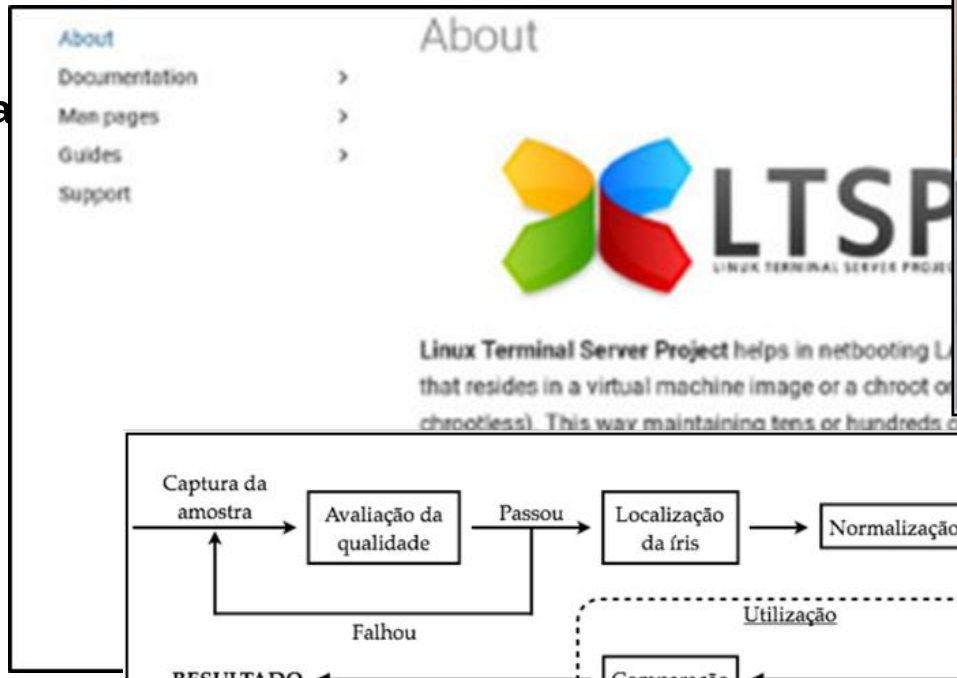


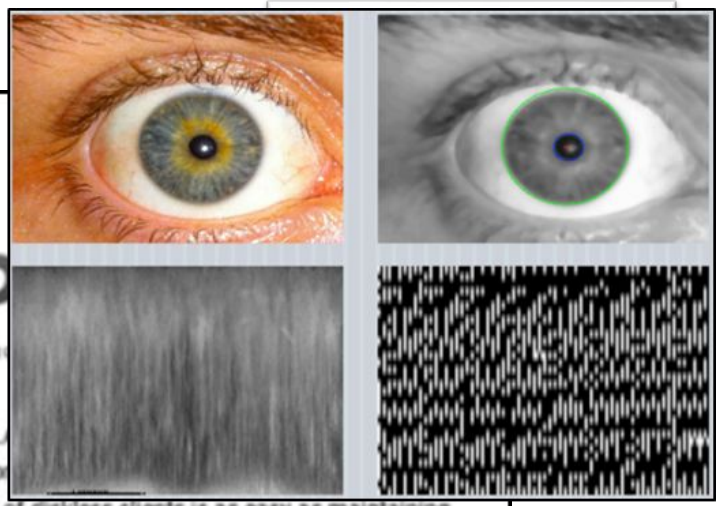

*Absolute Threshold for Vision*

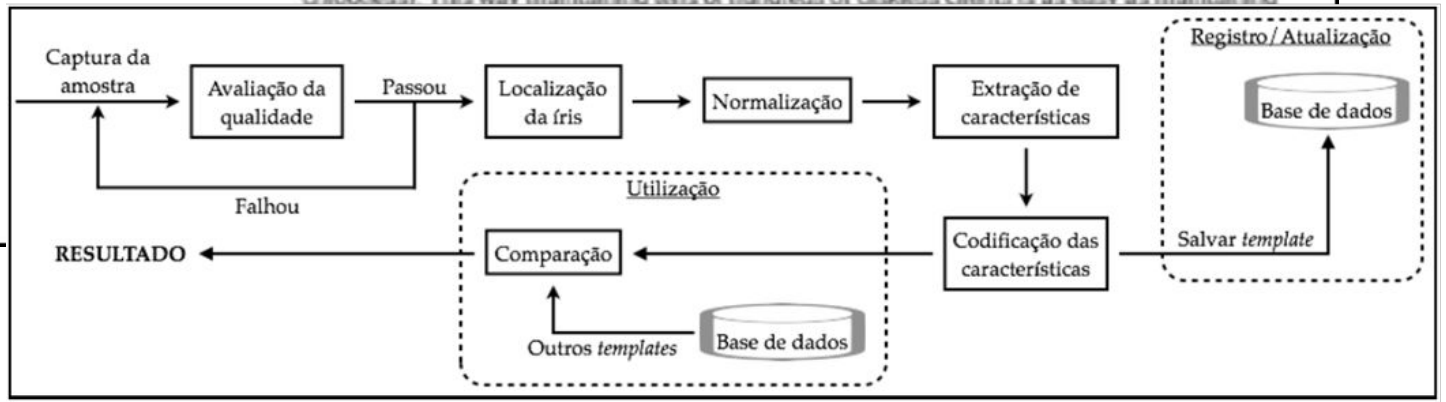
UniSENAI

# Sobre o professor

- Ma
  - 
  - 
  - 



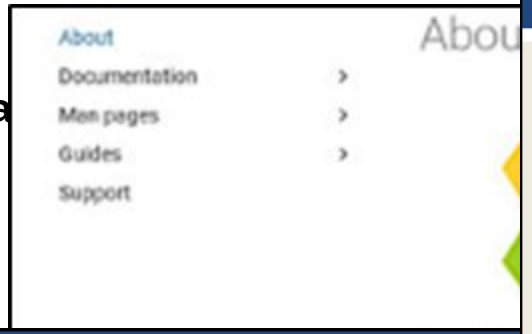



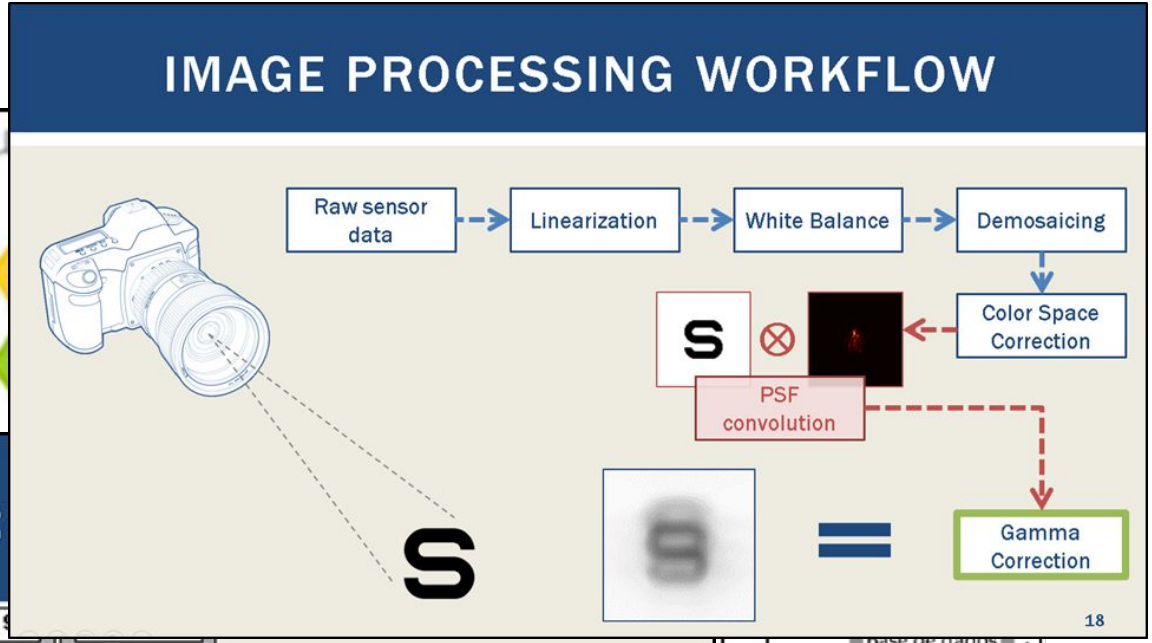


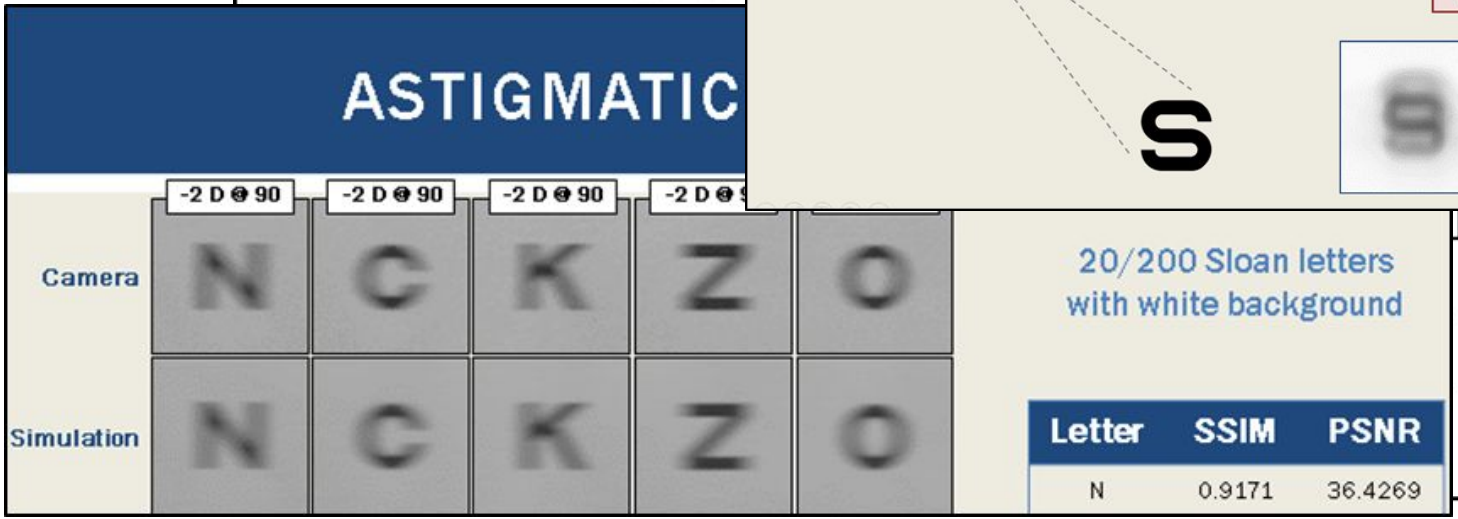
UniSENAI

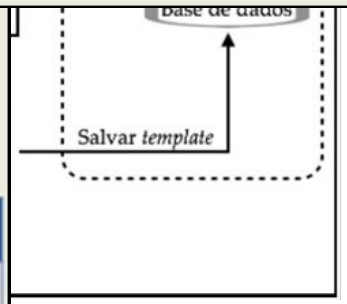
# Sobre o professor

- **Ma**
  - 



UniSENAI

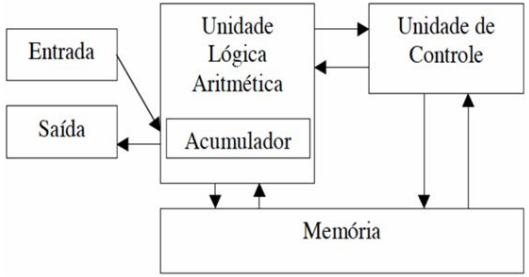
# Sobre os alunos

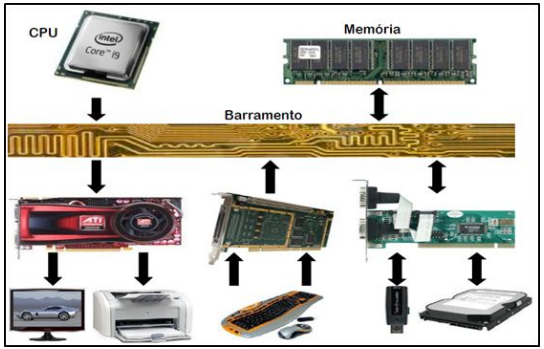
- **Expectativas** em relação à Unidade Curricular?
- Quais são suas **experiências prévias com estruturas de dados**? Pode citar cursos, projetos pessoais, experiência de trabalho ou qualquer outra atividade relacionada.
- Quais situações do seu dia a dia você percebe a **aplicabilidade de segurança da informação**?
- **Trabalha?** Em qual empresa / ramo?

UniSENAI

# Revisão de conceitos de computação

- Funcionamento do computador >>> **Máquina de Von Newmann**
  - Base dos computadores modernos
- **CPU**
  - A Unidade Central de Processamento, ou processador, é o "cérebro do computador". Responsável pelo controle geral das operações. Executa operações matemáticas e lógicas.
- **Entrada/Saída**
  - Por intermédio de dispositivos, sejam estes apenas de Entrada, apenas de Saída, ou ainda de Entrada/Saída, a comunicação entre os usuários/mundo exterior e o computador é possível.
- **Memória**
  - Responsáveis pelo armazenamento dos dados e programas.



UniSENAI

# Revisão de conceitos de computação

- **Representação básica dos dados**
- **BIT**
  - É a menor unidade de informação binária
  - Representado por 0 ou 1 → Ligado/Desligado, Verdadeiro/Falso, Sim/Não
  - Não é suficiente para representar, com significado, a maioria das informações
- **BYTE**
  - Composto por 8 bits
  - Espaço em memória são mensurados em Bytes e seus múltiplos
  - Podem representar informação (ex: Caracteres)
  - Os processadores tem a capacidade de processar "palavras" (unidades de informação) de 16, 32, 64, ..., bits por vez

UniSENAI

# Revisão de conceitos de computação

- **O funcionamento do computador**
- Basicamente, um computador funciona executando Operações Lógicas
- Antes de entender o que e quais são as operações lógicas, é importante salientar que um computador só "entende" duas situações:
  - *Com energia (ligado)*
  - *Sem energia (desligado)*
- Sistemas/bases numéricas importantes: **DECIMAL**, **BINÁRIO**, **HEXADECIMAL**
- Exemplo:
  - **15 em decimal**
  - **1111 em binário**
  - **F em hexadecimal**

UniSENAI

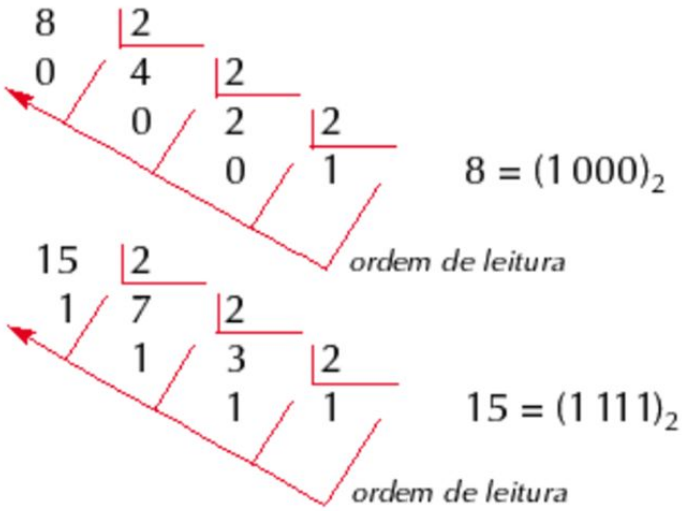
# Revisão de conceitos de computação

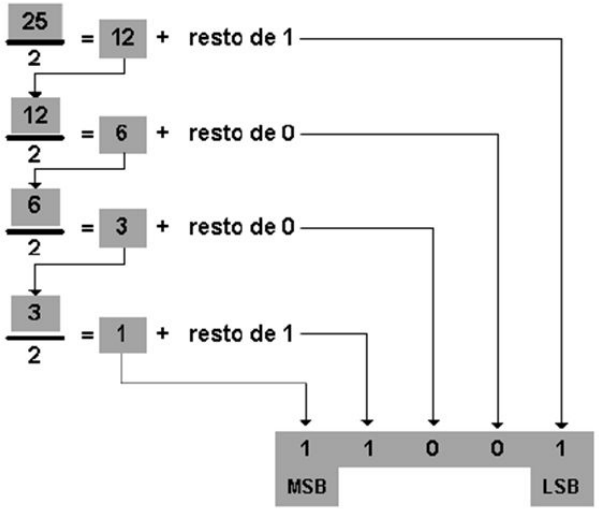
- **Decimal para Binário (base 10 para base 2)**
  - divisões sucessivas por 2
- **Binário para Decimal (base 2 para base 10)**
  - soma de potências de 2

UniSENAI

# Revisão de conceitos de computação

- **Base DECIMAL para BINÁRIO**



UniSENAI

# Revisão de conceitos de computação

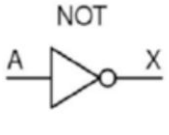
- **Base BINÁRIO para DECIMAL**

| 1 | 1 | 0 | 0 | 1 |
|---|---|---|---|---|
| $1x2^4$ | $1x2^3$ | $0x2^2$ | $0x2^1$ | $1x2^0$ |
| 16 + 8 + 0 + 0 + 1 = **25** | | | | |

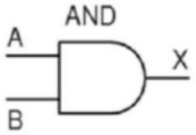
UniSENAI

# Revisão de conceitos de computação

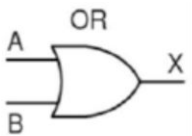
- O processador de um computador executa um conjunto de instruções e operações lógicas:
  - **NÃO (NOT)**
    - Só será verdadeiro (1) se ambas as entradas forem verdadeiras (1).
  - **E (AND)**
    - Será verdadeiro (1) sempre que uma das entradas for verdadeira.
  - **OU (OR)**
    - Se a entrada for verdadeira (1) a saída é falsa (0), e vice-versa.
  - **OU EXCLUSIVO (XOR)**
    - Ou com exclusividade



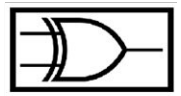

| A | X |
|---|---|
| 0 | 1 |
| 1 | 0 |




| A | B | X |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |




| A | B | X |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |



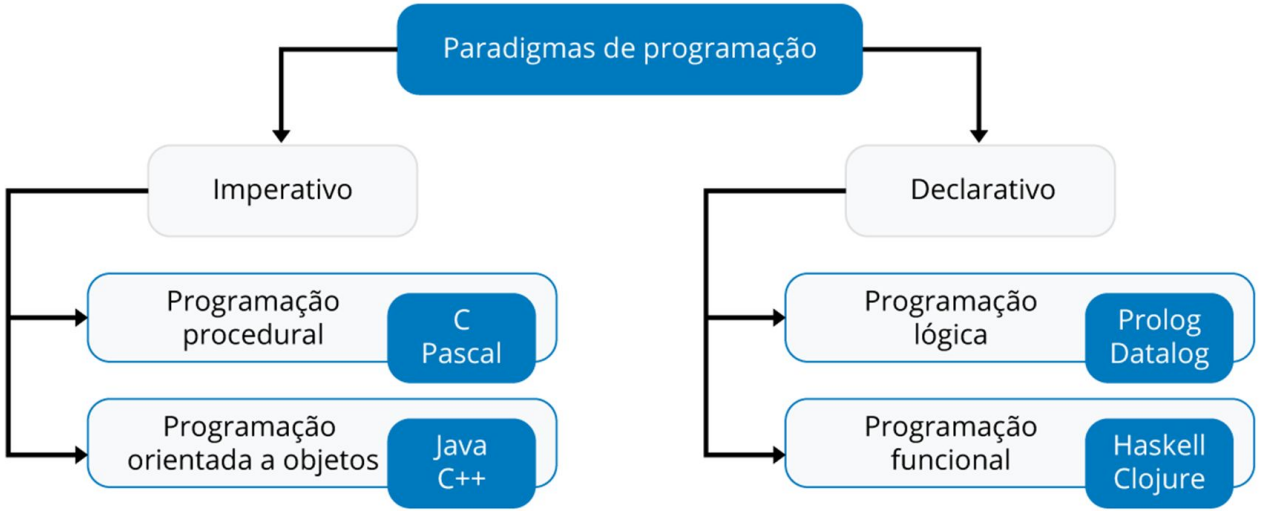
| A | B | XOR |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

UniSENAI

# Revisão de conceitos de programação

- **Paradigmas de programação**
  - Procedural / Estruturada
  - Orientado a Objetos
  - Funcional
  - Lógico

A **programação declarativa** é um paradigma de programação no qual é importante especificar o problema e o resultado esperado de sua solução



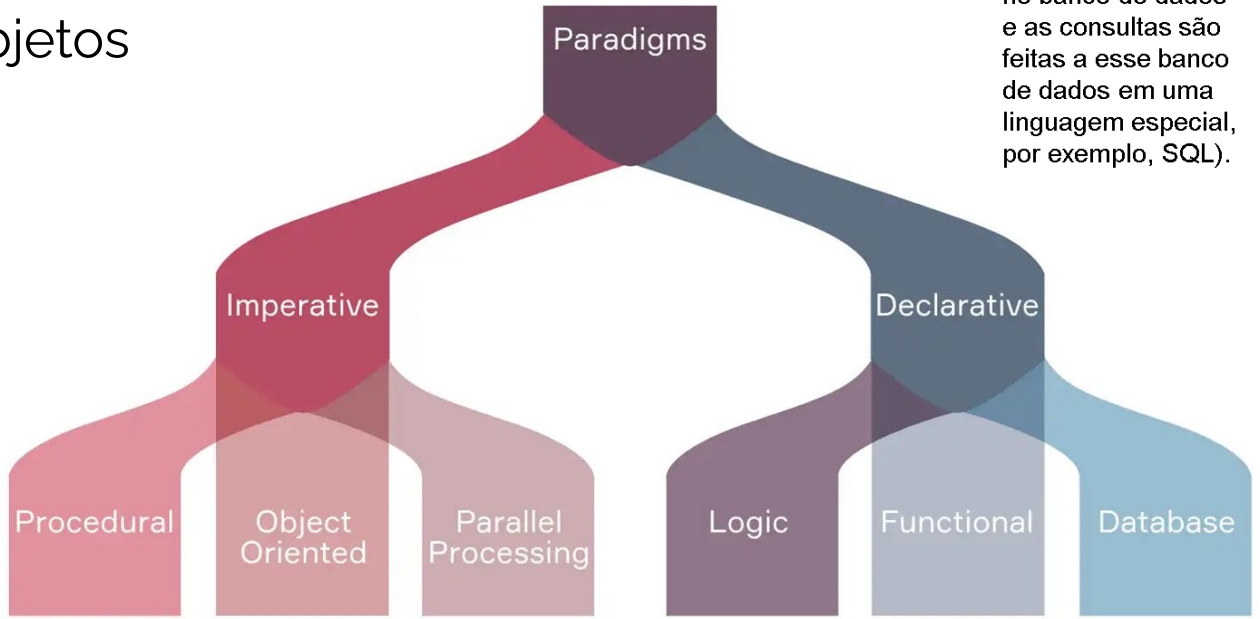

Um **programa imperativo** é semelhante às ordens expressas pelo modo imperativo nas línguas naturais.

UniSENAI

# Revisão de conceitos de programação

- **Paradigmas de programação**
  - Procedural / Estruturada
  - Orientado a Objetos
  - Funcional
  - Lógico

**Programação de Banco de Dados** é baseada no trabalho com dados (são armazenados no banco de dados e as consultas são feitas a esse banco de dados em uma linguagem especial, por exemplo, SQL).




O **processamento paralelo** ajuda a reduzir o tempo de execução da instrução, compartilhando ou paralelizando instruções entre vários processadores.

UniSENAI

# Revisão de conceitos de programação

## Estrutura básica de uma **classe**

*CONCEITO DE CLASSE SERÁ ESTUDADO NA UCr DE PROGRAMAÇÃO ORIENTADA A OBJETOS*

```
package <package_name>;

import <other_packages>;

public class NomeDaClasse {

    <variáveis>;
    <métodos construtores>;
    <outros métodos>;
}
```

- Uma classe Java está descrita em um arquivo de texto com uma extensão **.java**
- No exemplo mostrado, as **palavras-chave** Java estão destacadas em negrito
- A palavra-chave ***package*** define onde essa classe reside em relação a outras classes e fornece um nível de controle de acesso.
- A palavra-chave ***import*** define outras classes ou grupos de classes que você está usando na sua classe.
- A palavra-chave ***class*** precede o nome desta classe. O nome da classe deve corresponder ao nome do arquivo.

UniSENAI

# Revisão de conceitos de programação

## Uma classe/programa simples

*CLASSES, REFERÊNCIAS, MODIFICADORES DE ACESSO, RUNTIME, ETC. SÃO CONCEITOS QUE SERÃO ESTUDADOS NA UCr DE PROG OO*

```
public class MinhaPrimeiraClasse {

    public static void main(String args[]){

        System.out.println("Olá, mundo!");
    }
}
```

- Para executar um programa Java, é necessário definir um método **main** como mostra o slide. O método **main** é chamado automaticamente quando a classe é chamada na linha de comandos.
- Os argumentos de linha de comandos são especificados no programa por meio do array `args[]`.
- Observação: *Um método que é modificado com a palavra-chave static é chamado sem uma referência a um objeto específico. É utilizado o nome da classe. Esses métodos são chamados de métodos de classe. O método main é um método especial que é chamado quando esta classe é executada por meio do runtime Java.*

UniSENAI

# Revisão de conceitos de programação

## Blocos de Código

*Os blocos de código sempre são definidos com chaves { }.*
*Um bloco é executado pela execução de cada uma das instruções definidas no bloco, da primeira até a última (da esquerda para a direita).*

- Toda declaração de classe está contida em um **bloco de código**.
- As declarações de método estão contidas em **blocos de código**.
- Os campos e métodos Java têm escopo de bloco (ou classe).
- Os blocos de código são definidos entre chaves:

```
{        }
```

- Exemplo:

```
public class DigaUNISENAI {
    public static void main(String[] args) {
        System.out.println("UniSENAI");
    }
}
```

UniSENAI

# Revisão de conceitos de programação

## Comentários

- Os comentários no Java, como ocorre com os comentários na maioria das linguagens de programação, não são exibidos no programa executável.
- **3 maneiras de comentar código:**

```
// única linha
/* múltiplas linhas */
/ **  forma para gerar documentação */
```

UniSENAI

# Revisão de conceitos de programação

## Tipos de Dados Primitivos

- Tipos integrais (**byte**, **short**, **int** e **long**)
- Tipos de ponto flutuante (**float** e **double**)
- Tipo textual (**char**)
- Tipo lógico (**boolean**)

- Muitos dos valores usados em programas baseados na tecnologia Java são armazenados como tipos de dados primitivos. Acima estão dispostos os **oito tipos primitivos** incorporados à linguagem de programação Java.

UniSENAI

# Revisão de conceitos de programação

## Tipos primitivos integrais

| Tipo | Tamanho | Faixa | Exemplos de Valores Literais Permitidos |
|---|---|---|---|
| `byte` | 8 bits | de $-2^7$ a $2^7-1$ (de -128 a 127 ou 256 possíveis valores) | 2<br>-114<br>0b10 (número binário) |
| `short` | 16 bits | de $-2^{15}$ a $2^{15}-1$ (-32.768 a 32.767 ou 65.535 possíveis valores) | 2<br>-32699 |
| `int`<br>(tipo default para literais integrais) | 32 bits | de $-2^{31}$ a $2^{31}-1$ (-2.147.483.648 a 2.147.483.647 ou 4.294.967.296 possíveis valores) | 2<br>147334778<br>123_456_678 |

- Também é possível expressar quaisquer dos tipos integrais como um binário (0s e 1s). Por exemplo, uma expressão binária do número 2 é mostrada como um valor permitido do tipo integral **byte**. O valor binário é `0b10`. Observe que esse valor inicia com `0b` (ou seja, zero seguido da letra `B` minúscula ou maiúscula). Isso indica para o compilador que um valor binário virá em seguida.
- A capacidade de incluir sublinhados em um número **int** longo melhora a legibilidade do código. Esse recurso pode ser usado para facilitar a leitura de um número integral grande, substituindo-se as vírgulas por sublinhados. O uso do sublinhado não afetará o valor numérico do **int**, nem aparecerá se a variável for impressa na tela.

UniSENAI

# Revisão de conceitos de programação

## Tipos primitivos integrais

| Tipo | Tamanho | Faixa | Exemplos de Valores Literais Permitidos |
|---|---|---|---|
| `long` | 64 bits | $-2^{63}$ a $2^{63}-1$ (de -9.223.372.036854.775.808 a 9.223.372.036854.775.807 ou 18.446.744.073.709.551.616 possíveis valores) | 2<br>-20368547758O8L<br>1L |

- Ao especificar um valor literal para um tipo **long**, insira um L em maiúscula à direita do valor para declarar explicitamente que ele é um tipo **long**. O compilador considerará os literais inteiros como sendo do tipo **int**, a menos que você especifique de outra forma, usando um L para indicar o tipo **long**
- **Observação:** o único motivo para usar os tipos **byte** e **short** em programas é reduzir o consumo de memória. Como a maioria dos computadores desktop modernos contêm muita memória, a maioria dos programadores de aplicações para desktop não usa os tipos **byte** e **short**.

UniSENAI

# Revisão de conceitos de programação

## Tipos primitivos de ponto flutuante

| Tipo | Tamanho do Flutuante | Exemplos de Valores Literais Permitidos |
|---|---|---|
| `float` | 32 bits | 99F<br>-327456.99,01F<br>4,2E6F (notação de engenharia para $4,2 * 10^6$) |
| `double` (tipo default para literais de ponto flutuante) | 64 bits | -1111<br>2,1E12<br>99970132745699,999 |

- Acrescente F maiúsculo ou minúsculo ao número para especificar um número do tipo **float**.
- Os valores literais de tipos de ponto flutuante serão considerados como sendo do tipo **double**, a menos que você especifique de outra forma, usando a letra F para indicar o tipo **float**.
- **Observação:** use o tipo **double** quando houver necessidade de uma faixa maior ou de mais precisão.

UniSENAI

# Revisão de conceitos de programação

## Tipo primitivo textual

- O único tipo de dados primitivo textual é **char**.
- Ele é usado para um único caractere (16 bits).
- Exemplo:

```
char colorCode = 'U';
char simulaEnter = '\n';
```

- Ao designar um valor literal a uma variável **char**, como, por exemplo, t, você precisará colocar o caractere entre **aspas simples**: 't' O uso de aspas simples antes e depois do caractere informa ao compilador que t é apenas o valor literal t, e não uma variável t que representa outro valor.

UniSENAI

# Revisão de conceitos de programação

## Tipo primitivo lógico

- O único tipo de dados primitivo textual é **boolean**
- Capaz de armazenar somente um valor **true** ou **false**
- Exemplo:

```
boolean b1 = 2==1;
boolean b2 = (true || b1);
```

- As variáveis do tipo boolean podem armazenar somente os literais da linguagem de programação Java: *true* ou *false*

UniSENAI

# Revisão de conceitos de programação

## Valores padrão dos tipos primitivos

| Inteiro | Ponto Flutuante | Caractere | Verdadeiro Falso |
|---|---|---|---|
| byte<br>short<br>int<br>long | float<br>double | char | boolean |
| 0 | 0.0 | '\u0000' | false |

- O Java fornece quatro tipos de inteiro para acomodar números de diversos tamanhos. Todos os tipos numéricos têm sinal e, portanto, podem comportar números positivos ou negativos.
- Os tipos de ponto flutuante armazenam números com uma parte fracionária e obedecem ao padrão **IEEE 754**.

UniSENAI

# Revisão de conceitos de programação

## Literais Numéricos

```
public class NomeDaClasse {

    public static void main(String args[]){
       long creditCardNumber = 1234_5678_9012_3456L;
       long cpf = 123_456_789_00L;
       float pi =   3.14_15F;
       long hexBytes = 0xFF_EC_DE_5E;
       long hexWords = 0xCAFE_BABE;
       long maxLong = 0x7fff_ffff_ffff_ffffL;
       byte nybbles = 0b0010_0101;
       long bytes = 0b11010010_01101001_10010100_10010010;
    }
}
```

**Regras para Literais**

- É possivel inserir caracteres de sublinhado somente entre digitos. Não é possivel inserir esse caractere nos seguintes locais:
- No inicio ou fim de um número
- Adjacente a um ponto decimal em um literal de ponto flutuante
- Antes de um sufixo F ou L
- Em posições em que é esperada uma string de digitos
- *Observação: A linguagem Java faz distinção entre maiúsculas e minúsculas. Em Java, a variável creditCardNumber é diferente de CREDITCARDNUMBER. A convenção indica que os nomes de método e as variáveis Java usam "camelCase minúsculo" — minúscula na primeira letra do primeiro elemento de um nome de variável e maiúscula na primeira letra dos elementos subsequentes.*

UniSENAI

# Revisão de conceitos de programação

## Literais Binários

```
public class NomeDaClasse {

    public static void main(String args[]){

        // 8-bit 'byte':
        byte umByte = (byte)0b00100001;

        // 16-bit 'short':
        short umShort = (short)0b1010_0001_0100_0101;

        // 32-bit 'int':
        int umInt1 = 0b1010_0001_0100_0101_1010_0001_0100_0101;
        int umInt2 = 0b101;
        int umInt3 = 0B101; // o 'B' pode ser maiúsculo ou minúsculo
    }
}
```

**Regras para Binários**

- Também é possível expressar literais binários usando o sistema binário adicionando o prefixo 0b ou 0B ao número
- Literais binários são valores Java int. Os valores Java byte e short exigem uma conversão para impedir que o compilador emita um aviso de perda de precisão.

UniSENAI

# Revisão de conceitos de programação

## Nomeando uma variável

- **Regras:**
  - Os identificadores de variáveis devem iniciar com uma letra maiúscula ou minúscula, um sublinhado (_) ou um sinal de cifrão ($).
  - Os identificadores de variáveis não podem conter pontuação, espaços ou travessões.
  - As palavras-chave da linguagem Java não podem ser usadas
- **Diretrizes:**
  - Cada variável deve iniciar com uma letra minúscula. As palavras subsequentes devem começar com uma letra maiúscula (por exemplo, myVariable).
  - Escolha nomes mnemônicos e que indiquem o uso pretendido da variável para qualquer observador.
  - Assim como com uma classe ou método, um identificador ou um nome deve ser designado a cada variável do programa. Lembre-se de que **a finalidade da variável é servir como um mecanismo para o armazenamento e a recuperação de valores. Portanto, os seus identificadores devem ser simples, mas descritivos.**

UniSENAI

# Revisão de conceitos de programação

## Designando um valor a uma variável

- Exemplo:

```
double price = 12.99;
```

- Exemplo (booleana):

```
boolean isOpen = false;
```

- É possível designar um valor a uma variável no momento em que ela é declarada ou posteriormente. Para designar um valor a uma variável durante a declaração, adicione um sinal de igual (=) após a declaração, seguido do valor a ser designado.
- O operador = designa o valor à direita ao item à esquerda. O operador = deve ser lido como "é designado/atribuído a". No exemplo anterior, poderíamos afirmar que "o valor 12.99 é designado ao preço".
- **Observação:** os campos são inicializados automaticamente: os tipos integrais são definidos como **0**, os tipos de ponto flutuante são definidos como **0.0**, o tipo char é definido como **\u0000** e o tipo boolean é definido como **false**. Entretanto, você deve inicializar explicitamente os campos para que outras pessoas possam ler o código. As variáveis locais (declaradas em um método) devem ser inicializadas explicitamente antes de serem usadas.

UniSENAI

# Revisão de conceitos de programação

## Declarando e inicializando diversas variáveis em uma linha de código

- Sintaxe:

```
type identifier = value [, identifier = value];
```

- Exemplo (booleana):

```
double preco = 0.0, somaDosPrecos = 0.0;
```

- É possível declarar uma ou mais variáveis na mesma linha de código, mas **somente se todas forem do mesmo tipo**.

UniSENAI

# Revisão de conceitos de programação

## Outras maneiras de declarar variáveis e designar valores a elas

- Designando valores literais:

```
int ID = 0;
float pi = 3.14F;
char myChar = 'G';
boolean isOpen = false;
```

- Designando o valor de uma variável a outra:

```
int ID = 0;
int saleID = ID;
```

*A primeira linha de código cria um inteiro chamado **ID** e o utiliza para armazenar o número 0. A segunda linha de código cria outro inteiro chamado **saleID** e o utiliza para armazenar o mesmo valor como **ID** (0). Se o conteúdo de **ID** for alterado posteriormente, o conteúdo de **saleID** não mudará de forma automática. Embora os dois inteiros tenham o mesmo valor no momento, eles poderão ser alterados de forma independente em um programa posteriormente.*

UniSENAI

# Revisão de conceitos de programação

## Outras maneiras de declarar variáveis e designar valores a elas

- Designando o resultado de uma expressão a variáveis do tipo integral, ponto flutuante ou booleano.

```
float numberOrdered = 908.5F;
float casePrice = 19.99F;
float price = (casePrice * numberOrdered);
int hour = 12;
boolean isOpen = (hour > 8);
```

- Designando o valor de uma variável a outra:

```
int ID = 0;
int saleID = ID;
```

*A primeira linha de código cria um inteiro chamado **ID** e o utiliza para armazenar o número 0. A segunda linha de código cria outro inteiro chamado **saleID** e o utiliza para armazenar o mesmo valor como **ID** (0). Se o conteúdo de **ID** for alterado posteriormente, o conteúdo de **saleID** não mudará de forma automática. Embora os dois inteiros tenham o mesmo valor no momento, eles poderão ser alterados de forma independente em um programa posteriormente.*

UniSENAI

# Revisão de conceitos de programação

## Constantes

- Variável (pode ser alterada):

```
double salesTax = 6.25;
```

- Constante (não pode ser alterada):

```
final int NUMBER_OF_MONTHS = 12;
```

- **Diretriz:**
  - Nomeie as constantes de forma que elas possam ser facilmente identificadas. Em geral, as constantes devem iniciar em maiúscula, com as palavras separadas por um sublinhado (_).

- *Torne a variável uma constante usando a palavra-chave final para informar ao compilador que não deseja que o valor da variável seja alterado após ela ser inicializada. Se alguém tentar alterar o valor de uma constante após um valor ter sido designado a ela, o compilador emitirá uma mensagem de erro. Se você modificar o código a fim de designar um valor diferente à constante, precisará recompilar o programa.*

UniSENAI

# Revisão de conceitos de programação

## Operadores Matemáticos Padrão

| Finalidade | Operador | Exemplo | Comentários |
|---|---|---|---|
| Adição | + | `sum = num1 + num2;`<br>Se `num1` for igual 10 e `num2` for igual a 2, `sum` será igual a 12. | |
| Subtração | – | `diff = num1 - num2;`<br>Se `num1` for igual a 10 e `num2` for igual a 2, `diff` será igual a 8. | |
| Multiplicação | * | `prod = num1 * num2;`<br>Se `num1` for igual a 10 e `num2` for igual a 2, `prod` será igual a 20. | |
| Divisão | / | `quot = num1 / num2;`<br>Se `num1` for igual a 31 e `num2` for igual a 6, `quot` será igual a 5. | A divisão retorna um valor inteiro (sem resto). |

UniSENAI

# Revisão de conceitos de programação

## Operadores Matemáticos Padrão

| Finalidade | Operador | Exemplo | Comentários |
|---|---|---|---|
| Resto | % | `mod = num1 % num2;`<br>Se `num1` for igual a 31 e `num2` for igual a 6, `mod` será igual a 1. | Resto encontra o resto do primeiro número dividido pelo segundo número.<br>5 R<br>6 31<br>30<br>---<br>1<br>Resto sempre produz uma resposta com o mesmo sinal que o primeiro operando. |
| Atribuição | = | `int x = 2;` | Atribuição simples |
| Atribuição | += -= *= /= | `int x = 0;`<br>`x += 2;` | Atribuição com operadores aritméticos |

UniSENAI

# Revisão de conceitos de programação

## Operadores de Incremento e Decremento (++ e --)

| Operador | Finalidade | Exemplo | Observações |
|---|---|---|---|
| `++` | Pré-incremento<br>*(++variable)* | `int i = 6;`<br>`int j = ++i;`<br>`i vale 7, j vale 7` | |
| | Pós-incremento<br>*(variable++)* | `int i = 6;`<br>`int j = i++;`<br>`i vale 7, j vale 6` | O valor `i` é designado a `j` antes de `i` ser incrementado. Portanto, `j` recebe o valor `6`. |

| Operador | Finalidade | Exemplo | Observações |
|---|---|---|---|
| `--` | Pré-decremento<br>*(--variable)* | `int i = 6;`<br>`int j = --i;`<br>`i vale 5, j vale 5` | |
| | Pós-decremento<br>*(variable--)* | `int i = 6;`<br>`int j = i--;`<br>`i vale 5, j vale 6` | O valor `i` é designado a `j` antes de `i` ser decrementado. Portanto, `j` recebe o valor `6`. |

UniSENAI

# Revisão de conceitos de programação

## Precedência de Operadores

```
c = [[(25 - 5) * 4] / (2 - 10)] + 4;
c = [(20 * 4] / (2 - 10)] + 4;
c = (80 / (2 - 10)] + 4;
c = (80 / -8) + 4;
c = -10 + 4;
c = -6;
```

- Este é um exemplo em que são necessárias regras de precedência.
- A resposta do problema a seguir é **34** ou **9**?

  c = 25 - 5 * 4 / 2 - 10 + 4;

- **Regras de precedência:**
  - Operadores entre um par de parênteses
  - Operadores de incremento e decremento
  - Operadores de multiplicação e divisão, avaliados da esquerda para a direita
  - Operadores de adição e subtração, avaliados da esquerda para a direita
- Se operadores matemáticos padrão da mesma precedência aparecerem sucessivamente em uma instrução, eles serão avaliados da esquerda para a direita.

UniSENAI

# Revisão de conceitos de programação

## Operadores Relacionais e Lógicos

- **Operadores relacionais e de igualdade**
  - == Igual a
  - != Não igual a
  - > Maior que
  - >= Maior que ou igual a
  - < Menor que
  - <= Menor que ou igual a

- **Operadores lógicos**
  - && E condicional
  - || OU condicional
  - ~ NEGAÇÃO

UniSENAI

# Revisão de conceitos de programação

## Vetores

- **@DEVMEDIA**
  - Vetores são estruturas de dados que armazenam usualmente uma quantidade fixa de dados de um certo tipo; por esta razão, também são conhecidos como estruturas homogêneas de dados.
  - Internamente, um vetor armazena diversos valores, cada um associado a um número que se refere à posição de armazenamento, e é conhecido como índice. Os vetores são estruturas indexadas, em que cada valor que pode ser armazenado em uma certa posição (índice) é chamado de elemento do vetor

```
tipo identificador [= alocação de espaço]

int vetor[];
vetor = new int[10];

int outroVetor = new int[10];
```

UniSENAI

# Revisão de conceitos de programação

## Matrizes

- @DEVMEDIA
  - Segundo DEITEL (2005, pág. 204), um array é um grupo de variáveis (elementos ou componentes) que contém valores que são todos do mesmo tipo (estruturas homogêneas de dados). Em Java os arrays são objetos, portanto são considerados tipos por referência. Internamente, o que em geral considera-se um array é, na verdade, uma referência a um objeto array na memória. Pode-se encontrar como sinônimos para arrays: variáveis indexadas, variáveis subscritas, tabela de dados em memória ou matrizes (arrays bidimensionais).
  - Em variáveis (ou objetos) do tipo array podem ser armazenados um grupo de valores, cada um associado a um número ou números (no caso de estruturas multidimensionais) que se referem à posição de armazenamento. As posições do valor são conhecidas como índices ou subscritos do valor. Portanto, os arrays são estruturas indexadas, em que cada valor que pode ser armazenado em certa posição (índice) é chamado de elemento do array.

```
tipo identificador [= alocação de espaço]

int matriz[][];
matriz = new int[2][4];        // duas dimensões... linhas e colunas

int outraMatriz = new int[10][10];
```

UniSENAI

# Revisão de conceitos de programação

## Entrada e Saída no Java (*Input / Output*)

*Classes, objetos, construtores, métodos são tópicos da UCr de Programação OO*

```
import java.util.Scanner;

public class TestaDeclaracaoScanner {
        public static void main(String[] args) {
                // Lê a partir da linha de comando
                Scanner leitor = new Scanner(System.in);
                float numF =     leitor.nextFloat();
                int num1 =       leitor.nextInt();
                byte byte1 =     leitor.nextByte();
                long lg1 =       leitor.nextLong();
                boolean b1 =     leitor.nextBoolean();
                double num2 =    leitor.nextDouble();
                String nome =    leitor.nextLine();
                System.out.println("MENSAGEM");
        }
}
```

- A classe Scanner tem como objetivo separar a entrada dos textos em blocos, gerando os conhecidos *tokens*, que são sequências de caracteres separados por delimitadores que por padrão correspondem aos espaços em branco, tabulações e mudança de linha.
- É necessário criar um objeto do tipo Scanner que passa como argumento o objeto **System.in** dentro construtor (*o objeto System.in é o que faz a leitura do que se escreve no teclado*)
- Também é possível realizar a leitura utilizando o método `nextLine()`, que mostra a linha atual do objeto Scanner e avança para a próxima linha.

UniSENAI

# Revisão de conceitos de programação

## Exemplo01.java

```java
import java.util.Scanner;

public class Exemplo01 {
    public static void main(String args[]) {

      Scanner leitor = new Scanner(System.in);

      int numero=0;

      System.out.print("Informe um número: ");
      numero = leitor.nextInt();

      System.out.println("Você informou o número: " + numero);
    }
}
```

```
Informe um número: 12
Você informou o número: 12
```

UniSENAI

# Revisão de conceitos de programação

## Exemplo02.java

```
1  import java.util.Scanner;
2
3  public class Exemplo02 {
4      public static void main(String args[]) {
5
6        Scanner leitor = new Scanner(System.in);
7
8        String nome;
9
10       System.out.print("Qual o seu nome? ");
11       nome = leitor.nextLine();
12
13       System.out.println("Olá, " + nome + ", siga estudando...");
14     }
15 }
```

```
Qual o seu nome? Matheus
Olá, Matheus, siga estudando...
```

UniSENAI

# Revisão de conceitos de programação

## Exemplo03.java

```
1  import java.util.Scanner;
2
3  public class Exemplo03 {
4      public static void main(String args[]) {
5
6          Scanner leitor = new Scanner(System.in);
7
8          char sexo;
9
10         System.out.println("Informe um sexo...");
11         System.out.println("M ou m para masculino...");
12         System.out.println("F ou f para feminino...");
13         System.out.print("> ");
14         sexo = leitor.next().charAt(0);
15
16         if (sexo == 'm' || sexo == 'M') {
17           System.out.println("Sexo informado foi MASCULINO!");
18         } else {
19           if (sexo == 'f' || sexo == 'F') {
20               System.out.println("Sexo informado foi FEMININO!");
21           } else {
22               System.out.println("Sexo inválido...");
23           }
24       }
25   }
26 }
```

```
Informe um sexo...
M ou m para masculino...
F ou f para feminino...
> m
Sexo informado foi MASCULINO!
```

```
Informe um sexo...
M ou m para masculino...
F ou f para feminino...
> F
Sexo informado foi FEMININO!
```

```
Informe um sexo...
M ou m para masculino...
F ou f para feminino...
> X
Sexo inválido...
```

UniSENAI

# Revisão de conceitos de programação

## Exemplo04.java

```java
import java.util.Scanner;

public class Exemplo04 {
    public static void main(String args[]) {

        Scanner leitor = new Scanner(System.in);

        int valorInicial;
        int valorFinal;

        System.out.print("Informe um valor inicial: ");
        valorInicial = leitor.nextInt();
        System.out.print("Informe um valor final: ");
        valorFinal   = leitor.nextInt();

        System.out.println("Estes são todos os números inteiros no intervalo informado:");
        for (int i = valorInicial; i <= valorFinal; i++) {

            System.out.println(i);
        }
    }
}
```

```
Informe um valor inicial: 2
Informe um valor final: 17
Estes são todos os números inteiros no intervalo informado:
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
```

UniSENAI

# Revisão de conceitos de programação

## Exemplo05.java

```java
import java.util.Scanner;

public class Exemplo05 {
    public static void main(String args[]) {

        Scanner leitor = new Scanner(System.in);

        int opcao = 0;

        do {
            System.out.println("[1] Iniciar Jogo");
            System.out.println("[2] Configurações");
            System.out.println("[3] Créditos");
            System.out.println("[4] Sair");
            System.out.print("Qual a sua opção? ");
            opcao = leitor.nextInt();

            // código para tratar a opção escolhida pelo usuário

        } while (opcao != 4);
    }
}
```

```
[1] Iniciar Jogo
[2] Configurações
[3] Créditos
[4] Sair
Qual a sua opção? 1
[1] Iniciar Jogo
[2] Configurações
[3] Créditos
[4] Sair
Qual a sua opção? 4
```

UniSENAI

# Geração de um executável em Java

## Etapas para tradução híbrida do código-fonte Java e geração de executável

- Etapa 1 – escrever o programa
- Etapa 2 – compilar o programa (javac)
- Etapa 3 – executar o programa (java)
- Etapa 4 – criar um executável (jar)

UniSENAI

# Geração de um executável em Java

## Etapa 1 – escrever o programa

No Bloco de Notas do Windows, criar um arquivo e salvar como **MinhaPrimeiraClasse.java**

```
public class MinhaPrimeiraClasse {
    public static void main(String args[]) {
        System.out.println("Olá, SENAI!");
    }
}
```

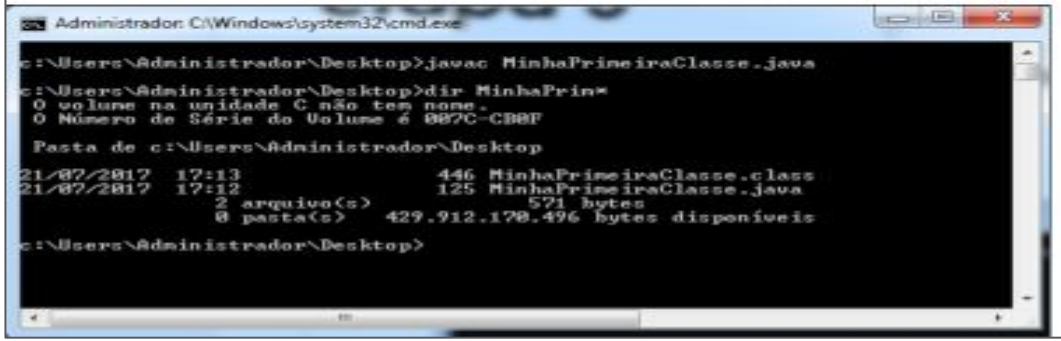
UniSENAI

# Geração de um executável em Java

## Etapa 2 – compilar o programa (javac)

Adicionar os arquivos executáveis do JDK no PATH do sistema operacional em questão (MS Windows)



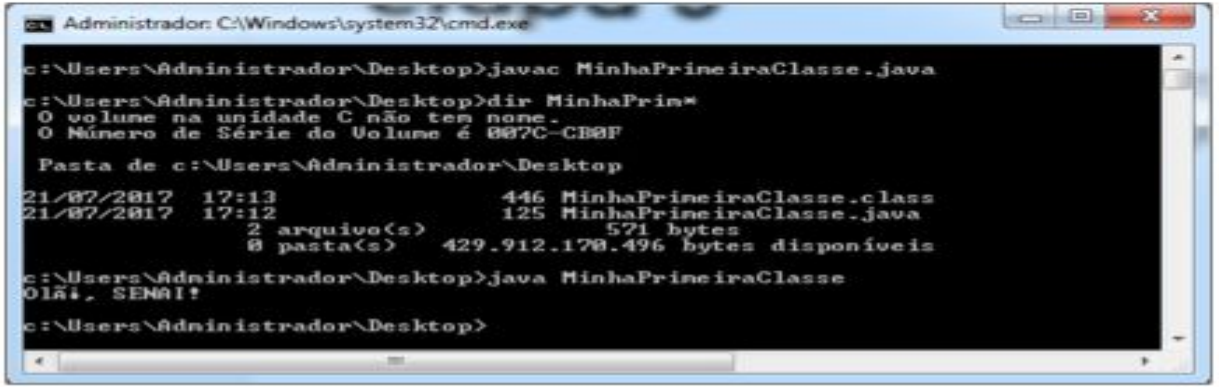

Compilar a classe **MinhaPrimeiraClasse.java**, criada na primeira etapa:

UniSENAI

# Geração de um executável em Java

## Etapa 3 – executar o programa (java)

Executar o *bytecode* da classe **MinhaPrimeiraClasse.java**, gerado na etapa anterior:



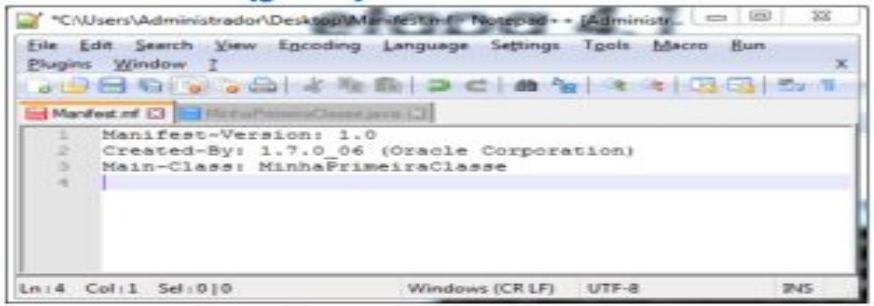
UniSENAI

# Geração de um executável em Java

## Etapa 4 – criar um executável (jar)

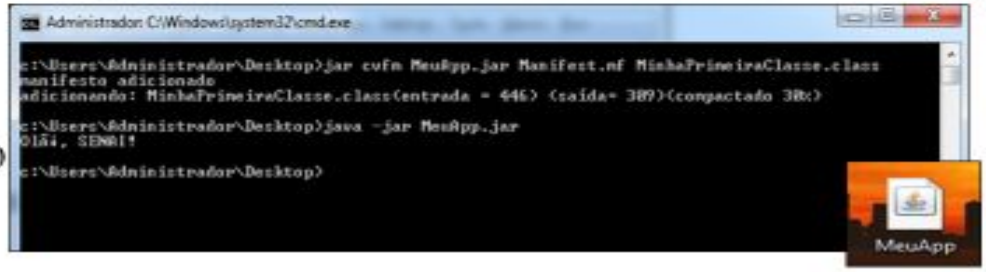
### *etapa 4.1*

Criar um arquivo de manifesto **"Manifest.mf"** para criação do executável java (JAR File):




### *etapa 4.2*

Gerar o arquivo executável (.jar), que será executado pela JVM do sistema operacional em questão:



UniSENAI

# Problemas de lógica de programação em Java

- **Momento mão na massa**
  - Crie um programa em Java utilizando os princípios da programação orientada a objetos. O programa deve conter uma classe chamada **Pessoa**, que possui os seguintes atributos: **nome** (tipo String), **idade** (tipo int), **peso** (tipo double) e **altura** (tipo double). Além disso, essa classe deve incluir métodos para **calcular o IMC da pessoa**, **verificar se ela é maior de idade** e **exibir um resumo com suas principais informações**. No **método principal (main)**, o programa deve solicitar ao usuário que digite os dados de uma pessoa, instanciar um objeto da classe Pessoa com essas informações e, em seguida, exibir o resumo da pessoa por meio de um método apropriado.

UniSENAI

# Problemas de lógica de programação em Java

- **Momento mão na massa**
  - Dando continuidade ao exercício anterior, amplie a classe **Pessoa** com novas funcionalidades. Adicione um método que **verifique se a idade da pessoa é um número par ou ímpar** e outro que **retorne o valor antecessor da idade**. Acrescente também um novo atributo chamado **saldoBancario** (do tipo double) e implemente um **método que retorne se o saldo é positivo, negativo ou igual a zero**. Além disso, adicione um atributo **salario** (também do tipo double) e um **método que permita reajustar esse salário em uma determinada porcentagem**, passada como parâmetro.
  - No **método principal (main)**, após exibir o resumo da pessoa (como feito na Parte 1), utilize os novos métodos para mostrar se a idade é par ou ímpar, qual é o antecessor da idade, qual a situação do saldo bancário (positivo, negativo ou zero) e o novo valor do salário após aplicar o reajuste com um valor informado pelo usuário.

UniSENAI

# Links interessantes

- https://www.oracle.com/br/java/
- https://academy.oracle.com/pt-br/oa-student.html
- https://www.jdoodle.com/online-java-compiler/
- https://glysns.gitbook.io/java-basico/

UniSENAI

# Oportunidade - English Online 2026 - British Council

- **Que tal começar o ano batendo a sua meta do ano de estudar Inglês?** 🗺️📚
- Se você está procurando fortalecer seu inglês e melhorar suas perspectivas de empregabilidade, o Banco Santander e o British Council estão oferecendo 10.000 vagas para que você possa fazer isso de casa e no seu próprio ritmo, durante 8 semanas. Além disso, se você for um dos 1.500 alunos com melhor desempenho, receberá 20 semanas adicionais de formação com aulas on-line com um professor. Inscreva-se agora! Ah! Você não precisa ter um diploma universitário nem ser cliente do Banco Santander.
- https://app.santanderopenacademy.com/pt-BR/program/english-online-2026-2nd-call

**Terá algum custo?** –

Sem nenhum custo. Os cursos são financiados pelo Banco Santander. Se desejar, ao final do curso, você poderá se inscrever por conta própria para o próximo nível, mas essa opção é totalmente pessoal e você não terá nenhuma pressão para isso.



UniSENAI

# Dúvidas? 😱

- **AVA** (https://ava.sesisenai.org.br/login/index.php)
- **E-mail** (matheus.krueger@edu.sc.senai.br)
- **WhatsApp** (https://wa.me/554733219647)

- **Monitoria**
  - Sábados, das 8h às 12h, na sala HUB.

*"A persistência é o caminho do êxito."*
*Charles Chaplin*